

Linear Regression: Ordinary Least Squares (OLS) Derivation

Quoc Kien

1. Introduction to Simple Linear Regression

In Simple Linear Regression, we model the relationship between a dependent variable y and an independent variable x using a straight line:

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

Where: * β_0 is the **intercept**. * β_1 is the **slope**. * $\hat{y}_i$ is the predicted value for the i -th observation.

Our goal is to find the optimal parameters β_0 and β_1 that minimize the difference between the actual observations y_i and our predictions $\hat{y}_i$.

2. The Loss Function (Residual Sum of Squares)

To measure the error of our model, we define the Objective/Loss Function $f(\beta_0, \beta_1)$ as half of the **Residual Sum of Squares (RSS)**. This scale factor $\frac{1}{2}$ is mathematically convenient because it cancels out later during differentiation:

$$f(\beta_0, \beta_1) = \frac{1}{2} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2 = \frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

To minimize this function, we take the partial derivatives with respect to β_0 and β_1 and set them equal to 0.

3. Deriving the Intercept (β_0)

First, let's take the partial derivative of f with respect to β_0 using the chain rule:

$$\frac{\partial f}{\partial \beta_0} = \sum_{i=1}^n -(y_i - \beta_0 - \beta_1 x_i) = 0$$

We can remove the negative sign by multiplying both sides by -1 :

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

Now, distribute the summation across the terms:

$$\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta_1 x_i = 0$$

Since β_0 is a constant, summing it n times equals $n\beta_0$:

$$\sum_{i=1}^n y_i - n\beta_0 - \beta_1 \sum_{i=1}^n x_i = 0$$

Isolate $n\beta_0$:

$$n\beta_0 = \sum_{i=1}^n y_i - \beta_1 \sum_{i=1}^n x_i$$

Divide the entire equation by n :

$$\beta_0 = \frac{1}{n} \sum_{i=1}^n y_i - \beta_1 \left(\frac{1}{n} \sum_{i=1}^n x_i \right)$$

Using the definition of sample means ($\bar{y} = \frac{1}{n} \sum y_i$ and $\bar{x} = \frac{1}{n} \sum x_i$), we get the final clean formula for the intercept:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

4. Deriving the Slope (β_1)

Next, we take the partial derivative of the loss function with respect to β_1 :

$$\frac{\partial f}{\partial \beta_1} = \sum_{i=1}^n -(y_i - \beta_0 - \beta_1 x_i)x_i = 0$$

Multiply by -1 and distribute x_i :

$$\begin{aligned} \sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2) &= 0 \\ \sum_{i=1}^n x_i y_i - \beta_0 \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 &= 0 \end{aligned}$$

Now, substitute our previously derived expression for β_0 ($\beta_0 = \bar{y} - \beta_1 \bar{x}$) into this equation:

$$\sum_{i=1}^n x_i y_i - (\bar{y} - \beta_1 \bar{x}) \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 = 0$$

Recognize that $\sum_{i=1}^n x_i = n\bar{x}$. Let's substitute that in:

$$\sum_{i=1}^n x_i y_i - (\bar{y} - \beta_1 \bar{x})(n\bar{x}) - \beta_1 \sum_{i=1}^n x_i^2 = 0$$

Expand the middle term:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} + n\beta_1 \bar{x}^2 - \beta_1 \sum_{i=1}^n x_i^2 = 0$$

Group the terms containing β_1 together on one side:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} = \beta_1 \sum_{i=1}^n x_i^2 - n\beta_1 \bar{x}^2$$

Factor out β_1 :

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} = \beta_1 \left(\sum_{i=1}^n x_i^2 - n\bar{x}^2 \right)$$

Finally, solve for β_1 :

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}$$

Alternative Algebraic Form (Covariance / Variance)

Using standard algebraic identities, this can also be beautifully expressed as:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

5. Summary of OLS Parameter Estimators

Parameter	Mathematical Formula	Statistical Meaning
Slope (β_1)	$\frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$	$\frac{\text{Cov}(X,Y)}{\text{Var}(X)}$
Intercept (β_0)	$\bar{y} - \beta_1 \bar{x}$	The baseline value of y when $x = 0$

6. Visualization Verification

Let's quickly run a verification script using Python to visualize how these closed-form solutions fit some dummy data points.

```
import numpy as np
import matplotlib.pyplot as plt

# Generate random dummy data
np.random.seed(42)
X = 2 * np.random.rand(50, 1)
y = 4 + 3 * X + np.random.randn(50, 1)

# Manual OLS Closed-form calculation
x_mean = np.mean(X)
```

```

y_mean = np.mean(y)

beta_1 = np.sum((X - x_mean) * (y - y_mean)) / np.sum((X - x_mean) ** 2)
beta_0 = y_mean - beta_1 * x_mean

# Generate prediction line
X_new = np.array([[0], [2]])
y_predict = beta_0 + beta_1 * X_new

# Plotting the results
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color="blue", label="Data Points")
plt.plot(X_new, y_predict, color="red", linewidth=2, label=f"OLS Line: y =
↪ {beta_0:.2f} + {beta_1:.2f}x")
plt.xlabel("Independent variable (x)")
plt.ylabel("Dependent variable (y)")
plt.title("OLS Linear Regression Fitting")
plt.legend()
plt.grid(True)
plt.show()

```

